

Biologically-Inspired Computation (F20BC) Coursework Report

LUKAS JURCAGA, JOAO LOPES

November 2023

Chapter 1 Introduction

In the area of artificial intelligence, the synergy between Artificial Neural Networks and Particle Swarm Optimization has emerged as a powerful paradigm for optimizing complex tasks. This report delves into the combination of ANN and PSO, exploring the intricate interplay between neural network architectures the optimization algorithm. The implementation section details the crafting of these intelligent systems, unraveling the design decisions behind classes and methods. The experimental investigation section analyses the hyperparameters governing the performance landscape and the delicate balance between complexity and efficiency. The results showcase the dynamic interplay of these components in achieving optimal configurations. The overarching goal is to yield insights into tailoring ANN architectures for this specific task, present the pivotal role of thoughtful hyperparameter selection. The report showcases the dynamic collaboration of ANN and PSO, pushing the boundaries of intelligent system design in the evolving AI landscape.

Chapter 2 Implementation

This section illustrates the implementation the Artificial Neural Network and Particle Swarm Optimization. It describes how the different classes and methods work, notes interesting aspects, and describes the motivation behind design decision.

2.1. ANN

2.1.1. Neuron

The ‘Neuron’ class is a fundamental component of an artificial neural network that encapsulates the behaviour of an individual neuron. During initialization, it receives the number of inputs and a chosen activation function. The class maintains internal fields for weights, bias, and the activation function. The weights are initialized randomly as an array, and the bias is set to a default value of ‘1.0’. The class provides methods to set the neuron’s weights and bias based on a given vector, retrieve the neuron’s state as a vector, calculate its output using the provided inputs, and obtain vector bounds for the PSO. Design decisions include setting bounds for PSO, initializing the bias to a non-zero value for symmetry avoidance, and representing the neuron’s state as a vector/matrix for simplicity and optimization ease.

2.1.2. Layer

The ‘Layer’ class, a vital element of the artificial neural network, is designed to manage a layer of neurons. During initialization, it receives the number of neurons, number of inputs per neuron, and an activation function. The class maintains a list of neurons and initializes each neuron with the specified inputs and activation function. Noteworthy aspects include the method to obtain the layer’s state as a vector, facilitating operations in optimization algorithms like PSO. The class can calculate the output of each neuron given input values, set neuron parameters based on a vector, and modify the activation function for the entire layer. Motivated by optimization considerations, the class provides vector bounds for weights and bias, crucial for optimization algorithms. The design prioritizes flexibility by allowing dynamic changes to neuron parameters

and activation functions. The class is instrumental in constructing and manipulating neural network layers efficiently.

2.1.3. ANN

The ‘ANN’ class orchestrates the construction and operation of an artificial neural network. It receives parameters during initialization, including the number of hidden layers, number of inputs, neurons per layer, and activation functions per layer. Notably, it dynamically constructs layers based on the specified parameters, fostering adaptability to diverse network architectures. The class employs a unique mechanism to map a continuous value to an activation function, allowing for a broader range of activation function choices. It adjusts the network parameters based on a given vector, showcasing a modular approach for optimization algorithms. The class calculates the network output and provides a vector representation of the entire ANN. Design decisions prioritize flexibility, encapsulation, and adaptability to varying activation functions and network structures. Additionally, the vector bounds provided support the PSO in maintaining feasible search spaces.

2.2. PSO

2.2.1. Particle

The ‘Particle’ class represents a particle in the Particle Swarm Optimization algorithm for optimizing the artificial neural network. It initializes with a position and velocity in the search space, maintaining information about its best position (cognitive component) and fitness. Unique aspects include a method that incorporate random restarts to prevent getting stuck in local optima. This class also integrates global and local information, facilitating exploration and exploitation. It promotes collaborative optimization by selecting the best position among informants and facilitating the sharing of information among particles. The class interfaces with an ANN for fitness evaluation, aligning with PSO’s goal of optimizing ANNs. The design emphasizes adaptability and collaboration among particles, addressing exploration-exploitation trade-offs.

2.2.2. Swarm

The ‘Swarm’ class manages a swarm of particles in the PSO algorithm for optimizing the ANN. It initializes with the number of particles, informants, and a landscape representing the search space. Each particle is created with a random position and velocity. Notably, the swarm utilizes a collaborative structure by assigning informants to each particle, enhancing information sharing. It synchronizes particle velocities and positions, reflecting PSO’s exploration-exploitation dynamics. Fitness assessment involves evaluating each particle’s fitness and updating the global best fitness and position. The collaborative nature of the swarm promotes efficient exploration of the solution space.

2.2.3. Landscape

The ‘Landscape’ class defines a search space within specified bounds for the PSO algorithm applied to the ANN. The class provides methods to generate random positions and velocities within the defined bounds, facilitating exploration. It calculates velocities by subtracting a random position from the current position, promoting dynamic movement within the landscape.

The class also includes methods for handling individual dimensions, checking bounds, and determining positions beyond the boundaries. The design decisions aim to create a versatile and dynamic landscape, encouraging diverse exploration strategies for the PSO algorithm. Default bounds were also encoded into the implementation of the ANN with weight values ranging from -1 to 1, bias from -5 to 5 and activation function from 0 to 1. These were chosen to reduce the problem space and restrict the particles from 'flying off'.

2.2.4. PSO

The 'PSO' class implements the PSO algorithm. It takes parameters such as the number of particles, informants, maximum iterations, and hyperparameters. An interesting aspect is the dynamic updating of hyperparameters during runtime, allowing flexibility in tuning the PSO algorithm. It initializes a swarm, iteratively evaluates fitness, and updates particle positions. Design decisions include encapsulating the PSO-specific logic in a separate class, allowing easy integration with different ANN structures. Additionally, the class provides methods to set the ANN and dynamically create a fitness landscape based on the ANN's vector bounds. The class returns the global best position, fitness, and current swarm positions after optimization.

2.3. Utilities

2.3.1. Activation Functions

The presented code defines an enumeration for the activation functions Sigmoid, ReLU, Tanh, and ELU. The dictionary maps each 'enum' to its respective function. It selects and applies the appropriate activation function based on the provided 'enum' and input. The code encapsulates activation functions, promoting reusability and clarity in the implementation of neural network layers.

2.3.2. ANN Evaluation

The provided code defines a utility module for evaluating an ANN using an accuracy metric. It computes the accuracy of the ANN's predictions compared to ground truth labels, considering a specified threshold. The interesting aspect is the use of modular functions for cost and accuracy metrics, enabling flexibility in evaluating different aspects of the ANN's performance.

Chapter 3 Experimental Investigation

In this experimental investigation, we delve into the optimization of Artificial Neural Networks (ANN) and Particle Swarm Optimization (PSO) hyperparameters. The pivotal role of hyperparameters in shaping the performance of these algorithms necessitates a meticulous exploration. Through systematic experimentation, we aim to discern the optimal configurations that enhance the predictive power and convergence efficiency of ANN-PSO synergies. For the full investigation see the Jupyter Notebook *Hyper_Parameter_Testing.ipynb* file under the testing directory.

3.1. ANN Hyperparameters

This report details the experimental investigation program conducted to optimize the ANN using the PSO algorithm. The investigation aimed to explore and refine various hyperparameters, including the number of layers, number of neurons per layer, and activation functions. The

objective was to enhance the predictive capabilities of ANNs for a binary classification problem based on the UCI banknote authentication dataset. The initial phase of the investigation concentrated on finding the optimal number of hidden layers. The number of neurons in each layer was set to one in these initial tests. The PSO hyperparameters were the same in each of these ANN tests which you can see in appendix G (from the Jupyter Notebook). After this, the best performing n-layer ANNs were chosen to be further investigated.

3.1.1. 2-Layer ANN

The decision to delve into 2-layer ANNs was motivated by the understanding that increasing complexity beyond this point might yield substantial improvements in accuracy. The hyperparameter of interest for the 2-layer ANN experiments was the number of neurons per layer. This investigation sought to understand the impact of layer depth on predictive accuracy. A range of 2 to 4 neurons per layer was chosen, aligning with the four inputs in the dataset. Throughout the experiments, PSO parameters remained constant to maintain consistency.

3.1.2. 3-Layer ANN

Building on the insights gained from the 2-layer experiments, the investigation extended to 3-layer ANNs. The focus shifted towards optimizing the number of neurons per layer in the hidden layers. Configurations with a single neuron per layer were excluded based on prior observations. The experiments with 3-layer ANNs aimed to unravel the influence of different configurations on performance. The emphasis was on understanding how variations in the number of neurons in the hidden layers affected the model's accuracy. Configurations such as 2-2 and 3-3 neurons in the hidden layers showed promising results, prompting further exploration. A subsequent test introduced even more complex configurations, featuring increased numbers of neurons (4-4 and 5-5). The objective was to discern the impact of heightened complexity on the model's performance.

The experimental investigation followed a methodical approach to optimize the ANN for binary classification using PSO. The choice of hyperparameters, such as the number of layers and neurons per layer, was guided by a balance between model complexity and the pursuit of optimal performance. This approach provides valuable insights into the process of tailoring ANN architectures for specific tasks, emphasizing the importance of thoughtful hyperparameter selection in the pursuit of optimal model performance.

3.2. PSO Hyperparameters

In the experimental investigation of PSO hyperparameters, a systematic approach was adopted to explore a range of values using grid search. The chosen hyperparameters for investigation included acceleration coefficients (alpha, beta, gamma, delta, epsilon), the number of particles, the number of informants, and the maximum number of iterations. These hyperparameters were selected based on their critical roles in influencing the performance and convergence of the Particle Swarm Optimization algorithm.

The first part of the investigation focused on tuning the acceleration coefficients. These coefficients play a key role in determining the particle movement and exploration-exploitation balance in the PSO algorithm. The chosen value ranges were carefully considered to encompass a variety of configurations that could impact convergence behaviour. For instance, alpha ranged from 0.4 to 0.9, beta from 1.5 to 2, gamma from 0 to 2, delta from 0 to 1.5, and epsilon from 0.5 to 1. These ranges were chosen due to their prevalence and success stated by multiple research papers [1, 2].

The second part of the investigation extended to testing the influence of the number of particles, the number of informants, and the maximum number of iterations. These parameters can significantly affect the diversity and convergence speed of the swarm. The selected ranges for these parameters were based on common practices and prior knowledge in the field. The number of particles was set at 50, and the number of informants at 4 for the first part. For the second part, the ranges were as follows: 30, 60, 80 for the number of particles; 3, 5, 8 for the number of informants; and 10, 30, 50, 100 for the maximum number of iterations. The testing structure followed a logical sequence.

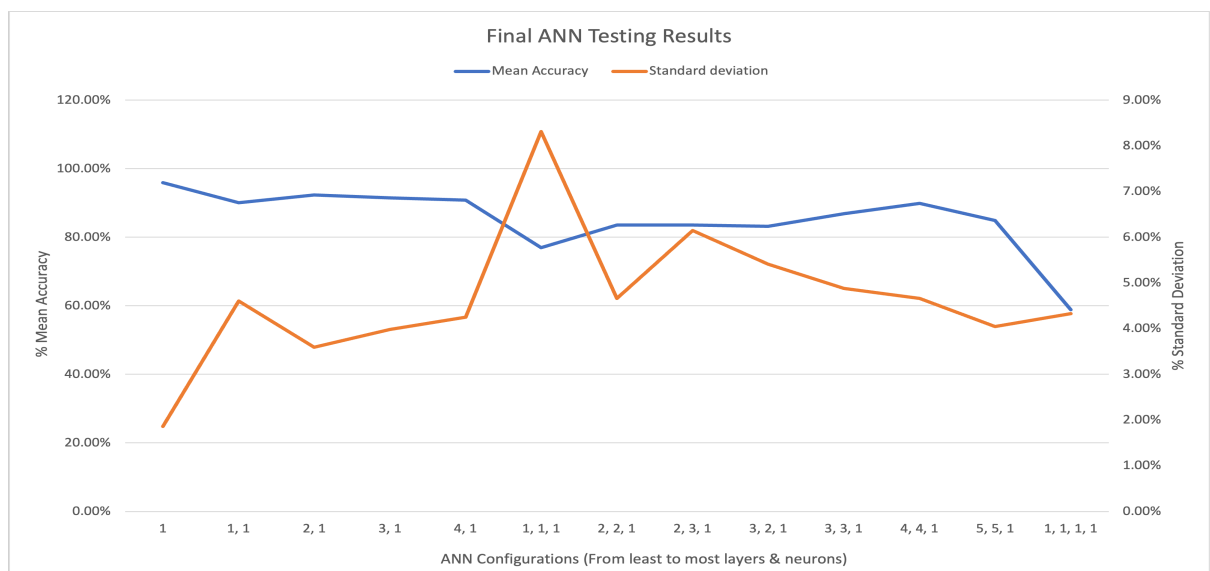
The initial focus on coefficients provided insights into the impact of individual hyperparameters. Subsequently, the best configurations from this phase were employed to further investigate the influence of swarm size, the number of informants, and iterations. The process aimed to progressively refine the hyperparameter settings based on the observed performance.

Overall, this experimental investigation employed a comprehensive grid search methodology, systematically exploring the hyperparameter space to identify configurations that achieve peak performance. This approach ensured a thorough examination of PSO hyperparameters, laying the foundation for any eventual future analysis and optimization.

Chapter 4 Results

4.0.1. ANN Hyperparameter Testing Results

The following is a line graph for the ANN testing results. The x axis represents the different ANN configures, where each number represents a separate layers with number being the amount of neuron in that layer. The last layer is always 1 as there is only one output neuron for a binary classification. For full ANN test results see appendix A.

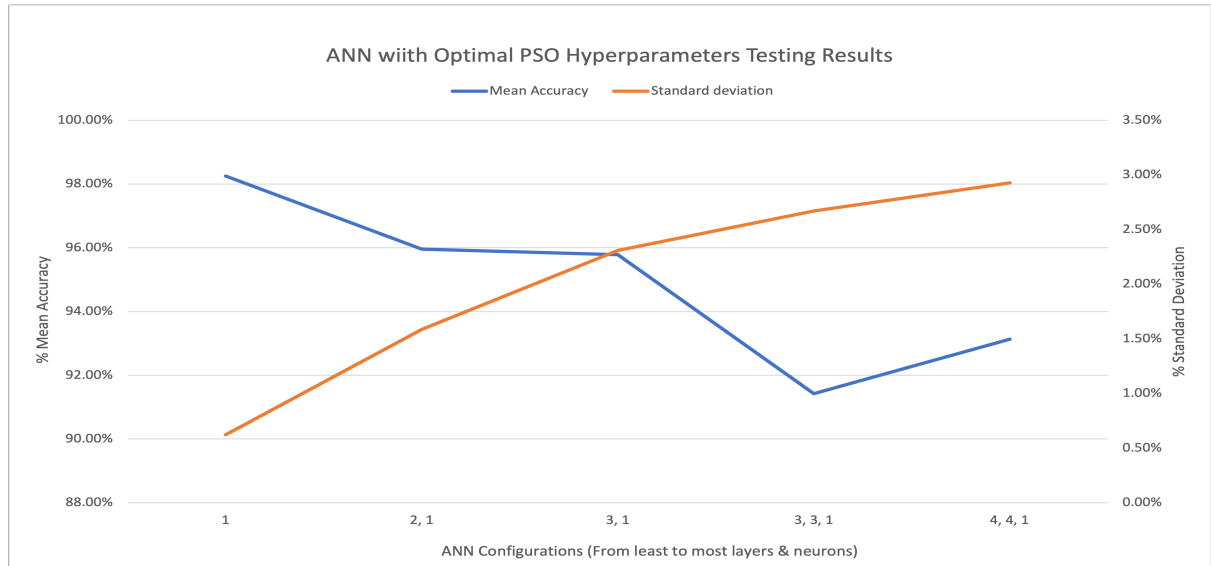


4.0.2. PSO Hyperparameter Testing Results

After running grid search through both stage 1 and stage 2, final ANN tests were run using the optimal PSO hyperparameters and results were plotted (see below). These ANN configurations were chosen as they had the best results from the previous round of testing. For the full results,

see appendix F.

In addition, two scatter graphs were also provided which shows the distribution of all the various permutations of hyperparameters that were tested in each of the stages. Each point on the graph represents a specific permutation of hyperparameters. See appendix C for stage 1 distribution and appendix E for stage 2 distribution. Full list of results for stage 1 and 2 have also been provided under appendix B and D respectively. These tables show all of the conducted tested and their PSO configuration.



Chapter 5 Discussion and Conclusions

The conclusive findings reveal that, in the realm of ANNs, bigger and more complex models don't always yield superior results. Surprisingly, during ANN testing, the top-performing model was the single-neuron ANN, closely followed by the two-layer ANN. This pattern persisted in PSO testing, where the more complex 2-layer and 3-layer ANNs failed to achieve the same accuracy levels as the single-neuron ANN, even with optimal PSO hyperparameters. This discrepancy may be attributed to the expansive search space associated with complex ANNs, hinting that the problem might be effectively addressed using a simpler logistic regression model.

In PSO testing, the optimal hyperparameters aligned with consensus in relevant research papers [1, 2]. The best-performing values for social aspects were 1.5 for the informant vector weight and 0 for the global best coefficient. Setting the global coefficient to 0 likely optimized performance by preventing premature convergence to local optima, as considering the overall swarm's best tended to induce this behavior. Focusing solely on informant best facilitated effective exploration while searching around potentially beneficial solutions.

The observations reveal that an increase in iterations or informant sizes doesn't always lead to better outcomes. Testing identified an optimal balance, suggesting that excessively large informant groups may limit exploration, leading to premature convergence to local optima. Conversely, overly small informant sizes may result in excessive exploration, hindering the search around already discovered good solutions.

References

- [1] Federico Marini and Beata Walczak. Particle swarm optimization (pso). a tutorial. *Chemo-metrics and Intelligent Laboratory Systems*, 149:153–165, 2015.
- [2] M Senthil Arumugam, MVC Rao, and Aarthi Chandramohan. A new and improved version of particle swarm optimization algorithm with global–local best parameters. *Knowledge and Information systems*, 16:331–357, 2008.

Appendix A ANN Testing Results

Number of layers	ANN Configuration	Mean Accuracy	Standard deviation
1	1	95.94%	1.86%
2	1, 1	90.04%	4.60%
2	2, 1	92.33%	3.59%
2	3, 1	91.48%	3.98%
2	4, 1	90.87%	4.25%
3	1, 1, 1	77.00%	8.31%
3	2, 2, 1	83.59%	4.66%
3	2, 3, 1	83.62%	6.15%
3	3, 2, 1	83.24%	5.41%
3	3, 3, 1	86.87%	4.88%
3	4, 4, 1	89.86%	4.66%
3	5, 5, 1	84.91%	4.05%
4	1, 1, 1, 1	58.87%	4.33%

Appendix B PSO Hyperparameter Testing Stage 1 Results

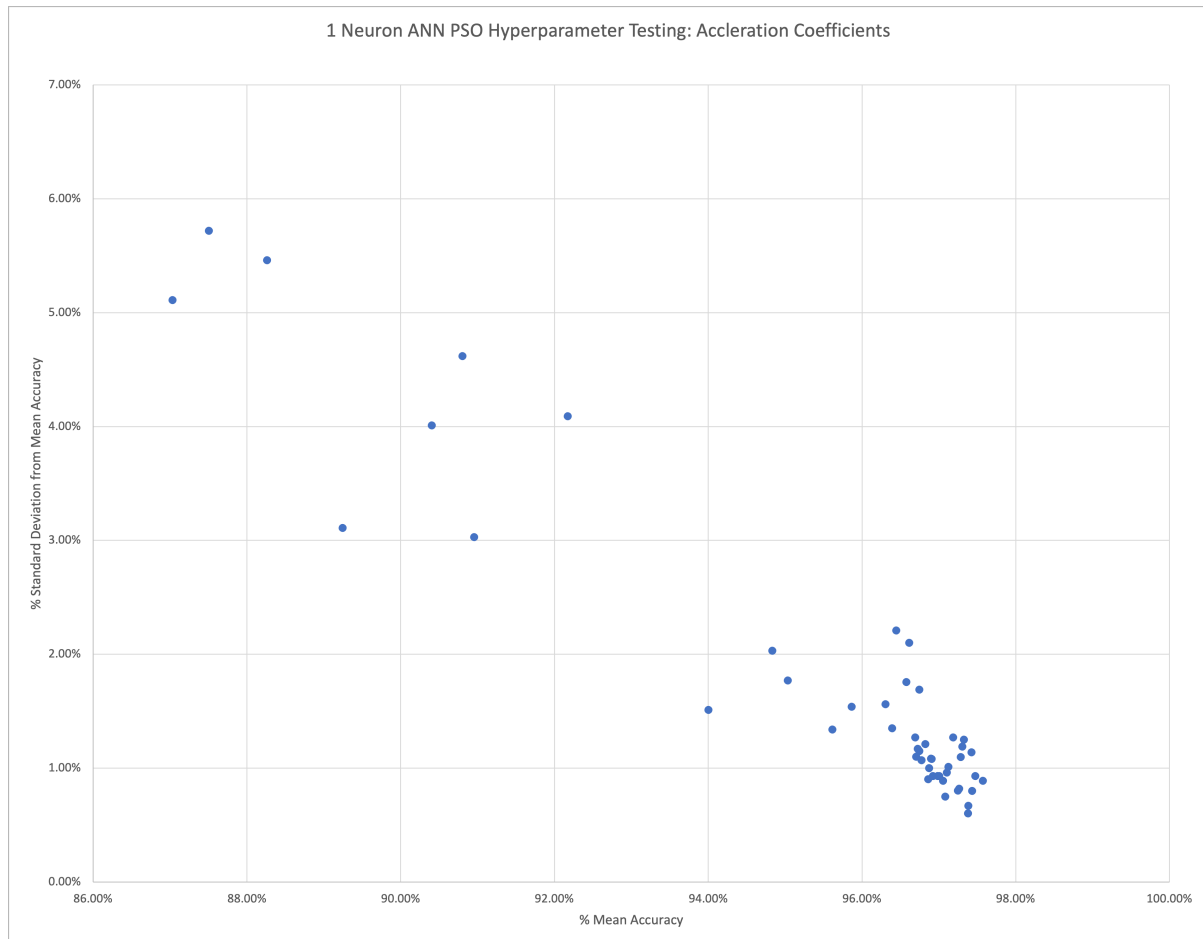
B.1. First Part

Test	Alpha	Beta	Gamma	Delta	Epsilon	Num Particles	Num Informants	Max Iter	Mean	Standard Deviation
1	0.4	1.5	0	0	0.5	50	4	50	0.90948	0.03027
2	0.4	1.5	0	0	1	50	4	50	0.87034	0.05115
3	0.4	1.5	0	1.5	0.5	50	4	50	0.90401	0.04010
4	0.4	1.5	0	1.5	1	50	4	50	0.92172	0.04094
5	0.4	1.5	1.5	0	0.5	50	4	50	0.97318	0.01247
6	0.4	1.5	1.5	0	1	50	4	50	0.97259	0.00824
7	0.4	1.5	1.5	1.5	0.5	50	4	50	0.96698	0.01105
8	0.4	1.5	1.5	1.5	1	50	4	50	0.96691	0.01265
9	0.4	1.5	2	0	0.5	50	4	50	0.97099	0.00959
10	0.4	1.5	2	0	1	50	4	50	0.96742	0.01689
11	0.4	1.5	2	1.5	0.5	50	4	50	0.96771	0.01074
12	0.4	1.5	2	1.5	1	50	4	50	0.95612	0.01343
13	0.4	2	0	0	0.5	50	4	50	0.87500	0.05722
14	0.4	2	0	0	1	50	4	50	0.90802	0.04615
15	0.4	2	0	1.5	0.5	50	4	50	0.88258	0.05463
16	0.4	2	0	1.5	1	50	4	50	0.89242	0.03105
17	0.4	2	1.5	0	0.5	50	4	50	0.97303	0.01193
18	0.4	2	1.5	0	1	50	4	50	0.97573	0.00889
19	0.4	2	1.5	1.5	0.5	50	4	50	0.96611	0.02099
20	0.4	2	1.5	1.5	1	50	4	50	0.95860	0.01544
21	0.4	2	2	0	0.5	50	4	50	0.97179	0.01272
22	0.4	2	2	0	1	50	4	50	0.96392	0.01354
23	0.4	2	2	1.5	0.5	50	4	50	0.97121	0.01012
24	0.4	2	2	1.5	1	50	4	50	0.96822	0.01210
25	0.9	1.5	0	0	0.5	50	4	50	0.96917	0.00932
26	0.9	1.5	0	0	1	50	4	50	0.96742	0.01155
27	0.9	1.5	0	1.5	0.5	50	4	50	0.94825	0.02026
28	0.9	1.5	0	1.5	1	50	4	50	0.96305	0.01555
29	0.9	1.5	1.5	0	0.5	50	4	50	0.97434	0.00798
30	0.9	1.5	1.5	0	1	50	4	50	0.96902	0.01079
31	0.9	1.5	1.5	1.5	0.5	50	4	50	0.96720	0.01170
32	0.9	1.5	1.5	1.5	1	50	4	50	0.97004	0.00934
33	0.9	1.5	2	0	0.5	50	4	50	0.97048	0.00892
34	0.9	1.5	2	0	1	50	4	50	0.97471	0.00932
35	0.9	1.5	2	1.5	0.5	50	4	50	0.96975	0.00930
36	0.9	1.5	2	1.5	1	50	4	50	0.97420	0.01136

B.2. Second Part

Test	Alpha	Beta	Gamma	Delta	Epsilon	Num Particles	Num In- formants	Max Iter	Mean	Standard Devia- tion
37	0.9	2	0	0	0.5	50	4	50	0.96436	0.02205
38	0.9	2	0	0	1	50	4	50	0.96873	0.01003
39	0.9	2	0	1.5	0.5	50	4	50	0.95029	0.01772
40	0.9	2	0	1.5	1	50	4	50	0.94001	0.0151
41	0.9	2	1.5	0	0.5	50	4	50	0.97077	0.00752
42	0.9	2	1.5	0	1	50	4	50	0.97383	0.00665
43	0.9	2	1.5	1.5	0.5	50	4	50	0.97376	0.00602
44	0.9	2	1.5	1.5	1	50	4	50	0.96859	0.00902
45	0.9	2	2	0	0.5	50	4	50	0.97281	0.01097
46	0.9	2	2	0	1	50	4	50	0.97245	0.00803
47	0.9	2	2	1.5	0.5	50	4	50	0.96895	0.01085
48	0.9	2	2	1.5	1	50	4	50	0.96574	0.01757

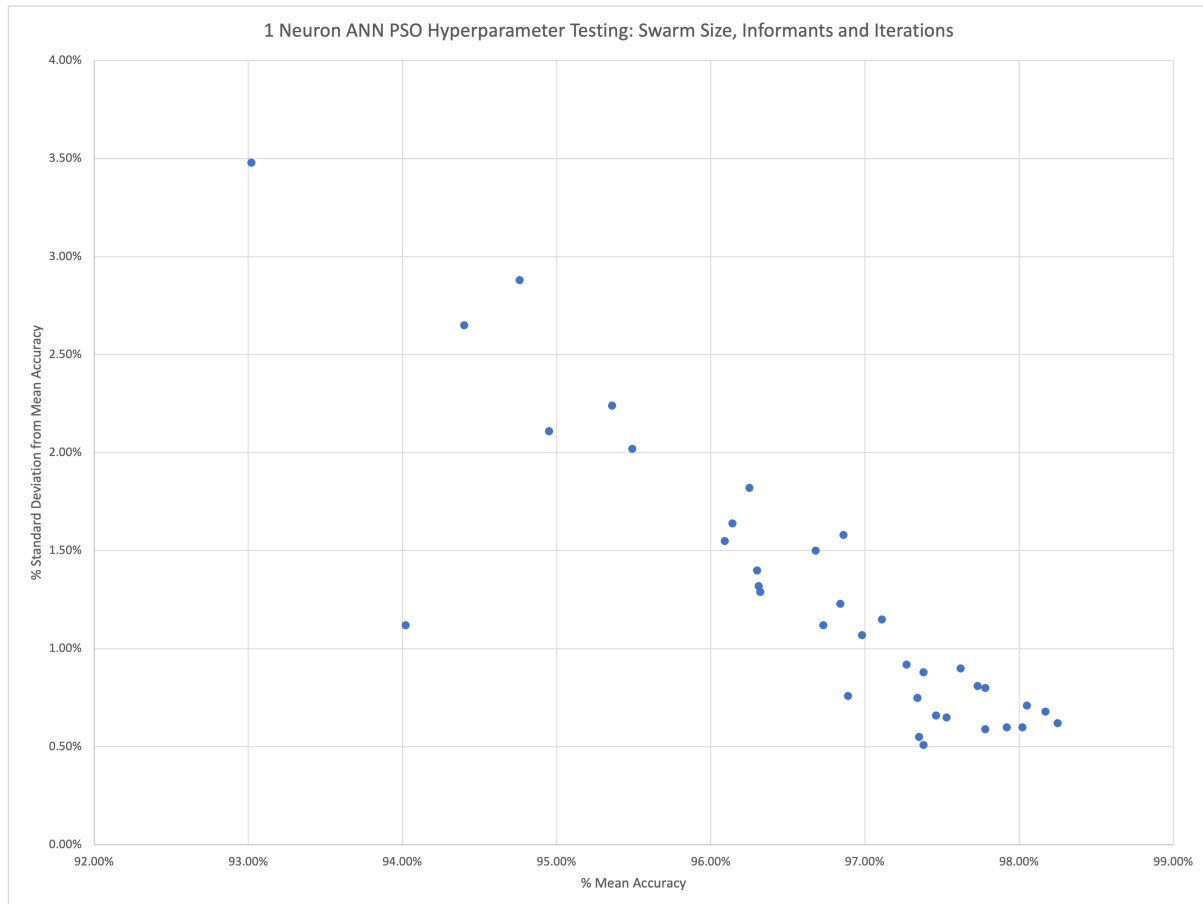
Appendix C Distribution of PSO Hyperparameter Testing Stage 1 Results



Appendix D PSO Hyperparameter Testing Stage 2 Results

Test	Alpha	Beta	Gamma	Delta	Epsilon	Num Parti- cles	Num Infor- mants	Max Iter	Mean	Standard Devia- tion
1	0.4	2	1.5	0	1	30	3	10	0.94949	0.02110
2	0.4	2	1.5	0	1	30	3	30	0.95488	0.02025
3	0.4	2	1.5	0	1	30	3	50	0.96859	0.01579
4	0.4	2	1.5	0	1	30	3	100	0.97347	0.00554
5	0.4	2	1.5	0	1	30	5	10	0.93025	0.03482
6	0.4	2	1.5	0	1	30	5	30	0.96246	0.01818
7	0.4	2	1.5	0	1	30	5	50	0.96312	0.01325
8	0.4	2	1.5	0	1	30	5	100	0.97529	0.00651
9	0.4	2	1.5	0	1	30	8	10	0.94395	0.02650
10	0.4	2	1.5	0	1	30	8	30	0.96319	0.01287
11	0.4	2	1.5	0	1	30	8	50	0.96975	0.01068
12	0.4	2	1.5	0	1	30	8	100	0.97383	0.00505
13	0.4	2	1.5	0	1	60	3	10	0.96305	0.01402
14	0.4	2	1.5	0	1	60	3	30	0.96727	0.01120
15	0.4	2	1.5	0	1	60	3	50	0.97456	0.00656
16	0.4	2	1.5	0	1	60	3	100	0.97777	0.00797
17	0.4	2	1.5	0	1	60	5	10	0.94759	0.02884
18	0.4	2	1.5	0	1	60	5	30	0.96895	0.00760
19	0.4	2	1.5	0	1	60	5	50	0.97617	0.00902
20	0.4	2	1.5	0	1	60	5	100	0.98025	0.00598
21	0.4	2	1.5	0	1	60	8	10	0.95364	0.02244
22	0.4	2	1.5	0	1	60	8	30	0.97106	0.01152
23	0.4	2	1.5	0	1	60	8	50	0.97376	0.00882
24	0.4	2	1.5	0	1	60	8	100	0.98047	0.00713
25	0.4	2	1.5	0	1	80	3	10	0.96144	0.01643
26	0.4	2	1.5	0	1	80	3	30	0.96837	0.01227
27	0.4	2	1.5	0	1	80	3	50	0.97777	0.00594
28	0.4	2	1.5	0	1	80	3	100	0.97923	0.00599
29	0.4	2	1.5	0	1	80	5	10	0.96093	0.01546
30	0.4	2	1.5	0	1	80	5	30	0.97274	0.00925
31	0.4	2	1.5	0	1	80	5	50	0.98251	0.00624
32	0.4	2	1.5	0	1	80	5	100	0.97726	0.00813
33	0.4	2	1.5	0	1	80	8	10	0.94016	0.01118
34	0.4	2	1.5	0	1	80	8	30	0.96684	0.01496
35	0.4	2	1.5	0	1	80	8	50	0.97340	0.00753
36	0.4	2	1.5	0	1	80	8	100	0.98171	0.00677

Appendix E Distribution of PSO Hyperparameter Testing Stage 2 Results



Appendix F ANN Testing with Optimal PSO Hyperparameters Results

Number of layers	ANN Configuration	Mean Accuracy	Standard deviation
1	1	98.25%	0.62%
2	2, 1	95.95%	1.59%
2	3, 1	95.78%	2.31%
3	3, 3, 1	91.42%	2.67%
3	4, 4, 1	93.13%	2.93%

Appendix G Default PSO Hyperparameters During the Initial ANN testing Phase

ANN Hyperparameter Testing

In this section, the ANN parameters were the independent variables. These were manipulated and changed in each run to find the optimal value for each ANN parameter. For each of the experiments, the following base PSO parameters were used:

- Alpha: 0.8
- Beta: 2
- Gamma: 2
- Delta: 0 (*Usually preferable to not use global best due to early convergence on local optima*)
- Epsilon: 1
- Number of Informants: 5 (*Random*)
- Number of Particles: 30 (*Reduces computational complexity while also maximising search effectiveness*)
- Maximum Number of Iterations: 25 (*Time restriction*)

```
pso = Pso(ann=None, num_particles=30, num_informants=5,   
max_iter=25, alpha=0.8, beta=2, gamma=2, delta=0, epsilon=1)
```

Executed at 2023.11.20 16:28:27 in 1ms